Limitless Deal Hub project proposal

Interactive car showroom and Three.js driving experience

Author: Durim Memishi

Subject: Computer Graphics

Date: 5 November 2026

Proposal summary

This project proposes an interactive web showroom in which visitors can inspect a vehicle in 3D, change its appearance and drive their design through a virtual city. The project connects computer graphics concepts to a complete application with accounts, saved configurations and database persistence. An implemented prototype already provides the core scope described here. This proposal defines its academic purpose, evaluation criteria and further development.

The intended result is a demonstrable graphics application supported by clear documentation and repeatable checks. The project does not aim to prove sales growth or replace a physical test drive. Its prices and checkout are for demonstration, and its driving system uses simplified arcade physics.

Problem statement

A catalog of static images offers only the viewpoints and colors chosen in advance. An interactive model lets the visitor inspect the same object from different angles and observe how material and lighting choices affect its appearance. A driving scene extends this interaction by placing the visitor's configuration in an animated environment.

For a Computer Graphics project, this creates a useful setting for demonstrating scene composition, transforms, surface materials, lighting, camera movement, animation and input handling. Account persistence adds a practical application layer around those graphics features.

Aim and objectives

The aim is to build and evaluate a responsive 3D vehicle showroom with persistent user designs.

1. Present a catalog of 12 vehicles across nine brands with search, filtering and comparison.
2. Render locally hosted GLB models with interactive cameras and an assembly reveal.
3. Keep body paint, wheel finish and glass customization independent and preserve the selected design.
4. Provide keyboard and touch control in a city containing roads, traffic, pedestrians and an animated day and night transition.
5. Store account profiles, named designs and account carts through an authenticated backend.
6. Document setup, deployment, asset attribution and verification so another student or examiner can reproduce the demonstration.

Scope and intended users

The primary audience includes car enthusiasts exploring an educational showroom, students studying web graphics and an examiner reviewing the implementation. Guests can browse, compare, customize and drive vehicles. Registered users gain profile editing, named saved designs and a persistent account cart.

Functional scope

The website includes responsive navigation, an automatically moving brand logo carousel, vehicle details, a material editor, a device shortlist and comparison of up to three vehicles. The graphics experience includes a 3D presentation area and a separate controllable city game. The game supports camera views, animated wheels, daytime changes and a long acceleration corridor.

The account system supports registration, sign-in, sign-out, profile images, named designs and saved carts. The backend validates ownership and normalizes incoming configurations. Local development uses SQLite, while the Vercel deployment uses Neon PostgreSQL.

Scope boundaries

Real purchasing, payment processing, vehicle reservations, manufacturer-authorized CAD accuracy and a physically complete driving simulator are outside the current scope. Multiplayer, route planning, real-world traffic data and an administration portal are also outside the prototype. Account recovery and email verification remain future work.

Technical approach

React and React Router provide the page structure and interface state. Vite manages local development and production builds. Three.js provides the graphics engine, React Three Fiber connects the scene to React and Drei supplies common scene utilities. GSAP handles selected interface animation.

Vehicle metadata maps catalog IDs to model files and licensed previews. The rendering pipeline prepares geometry and maps model materials to editable roles. The wheel rig groups wheel geometry around pivots so speed and steering can animate it. A time-stepped vehicle update handles the player's movement and basic collisions.

The frontend sends same-origin requests to a Node-based API. Local development uses a Node HTTP server and SQLite. Production uses a Vercel function and PostgreSQL through pg. Password hashes, session tokens, parameterized queries and ownership checks support protected account operations. Development must review these controls further before handling a commercial service.

Data design

The database contains users, sessions, carts and designs. Production also includes persistent login attempt counters. Saved design records contain a vehicle ID and normalized configuration. Browser drafts and shortlists remain device-local, separate from account-owned records. The project must preserve this distinction in both interface text and documentation.

Development plan

Four phases organize the continuing development and review. The current prototype already implements substantial parts of each phase.

Phase one requirements and assets

Confirm user journeys and scope, review model licenses, define the vehicle catalog and prepare a source structure. Acceptance requires a route list, usable model files and attribution for each asset.

Phase two graphics and interaction

Implement model viewing, material roles, lighting presets and wheel articulation. Add vehicle input, cameras, city movement and the day and night transition. Acceptance requires a customized design to retain independent paint and wheel values when entering the drive.

Phase three persistence and deployment

Connect accounts, profiles, saved designs and carts to the local and cloud backends. Validate session handling and record ownership. Deploy through GitHub and Vercel with a persistent Neon database. Acceptance requires saved account data to return after sign-out and sign-in, and the deployed health endpoint to report a ready database.

Phase four evaluation and delivery

Run regression checks, review responsive layouts and complete a physical device test where possible. Prepare the technical guide, a project proposal and a five to ten minute presentation. Record known limitations and prioritize the next changes based on observed failures.

Evaluation criteria

- Coverage: all 12 catalog entries have valid IDs, previews and matching model metadata.
- Customization: changing wheel finish preserves body paint, and saved configuration data survives the intended storage round trip.
- Graphics: the viewer loads, cameras respond, front wheels steer and wheels rotate while driving.
- Simulation: braking works while throttle is held, steering returns smoothly and tested narrow obstacles block movement.
- Persistence: one account cannot request another account's records through supported API operations. Cart and saved designs remain available after signing in again.
- Responsiveness: navigation, customization and comparison remain usable at desktop and phone viewport sizes. Touch driving controls remain reachable.
- Reproducibility: a fresh local installation starts both services using the documented commands, and the production build succeeds.

The baseline contains 13 passing automated regression checks and six model integration checks. These are evidence of selected behaviors, not proof of complete correctness. Further API integration tests, security review, accessibility review and device measurements remain part of evaluation.

Resources and feasibility

The project requires a development computer, a browser with WebGL support, VS Code, Node.js and Git. GitHub stores the source, Vercel hosts the application and Neon supplies the deployed database. Hosting use must stay within the account's chosen plan and current allowances. This proposal makes no fixed hosting cost or ongoing free-service guarantee.

Existing models and logos reduce modeling effort, but they require licensing review and accurate credit notices. The work focuses on integrating, adapting and interacting with these assets. It does not claim original authorship of third-party vehicle models.

Main risks and responses

Model complexity and slow loading

Large meshes, textures and device resolution can reduce responsiveness. Use compressed assets, defer heavy scenes where practical and retain Lite graphics. Measure real loading and frame rates before choosing further optimizations.

Material and wheel incompatibility

Model authors use different names and mesh structures. Maintain explicit material mappings, normalize axes and verify four wheel pivots. Keep a manual part-selection fallback and a regression case for independent finishes.

Data loss and deployment mistakes

Keep local database files private, back up before schema changes and preserve the intended production database branch. Review deployments and health checks. Add versioned migrations before substantial changes to the schema.

Security and account recovery

Use the implemented validation and ownership checks, then extend evaluation with API integration tests and an independent review. Add email verification and password recovery before presenting the service as ready for general commercial use.

Asset rights and accuracy

Preserve authorship, licenses and modification notes. Some existing models restrict commercial use, and some model trims differ from catalog specifications. Review each asset before commercial distribution and retain the visible model notes.

Deliverables

The deliverables are the running website and city game, the GitHub source, the database-backed account features, technical and user documentation, this proposal, and an English presentation with a timed demonstration guide. The project will be assessed against the stated acceptance criteria and its explained limitations.

Project references

Live application: https://comp-graph-project.vercel.app/

Source baseline: https://github.com/Dm31650/comp-graph-project/tree/14b42a9

Supporting evidence includes the repository's package.json, src/, server/, tests/ and model and logo credit notices under public/.